Sieve: Dynamic Expert-Aware PIM Acceleration for Evolving Mixture-of-Experts Models

Jungwoo Kim
Stanford University
Stanford, California, USA
jungwkim@stanford.edu

Rubens Lacouture
Stanford University
Stanford, California, USA
rubensl@stanford.edu

Genghan Zhang
Stanford University
Stanford, California, USA
zgh23@stanford.edu

Gina Sohn
Stanford University
Stanford, California, USA
ginasohn@stanford.edu

Qizheng Zhang
Stanford University
Stanford, California, USA
qizhengz@stanford.edu

Swapnil Gandhi
Stanford University
Stanford, California, USA
gandhis@stanford.edu

Christos Kozyrakis
Stanford University
Stanford, California, USA
NVIDIA
Santa Clara, California, USA
kozyraki@stanford.edu

Kunle Olukotun
Stanford University
Stanford, California, USA
kunle@stanford.edu

Abstract

Mixture-of-Experts (MoE) has become a dominant architecture for scaling large language models (LLMs). However, the execution characteristics of MoE inference are changing rapidly and increasingly mismatch the assumptions underlying existing Processing-in-Memory (PIM) systems. Prior PIM systems for LLMs rely on static rules to offload memory-bound operations to PIM, without accounting for the combined effects of load imbalance and inter-GPU communication. Meanwhile, modern MoE models activate fewer experts out of increasingly many, creating a bimodal expert distribution: a small set of experts receives many tokens, while a long tail of experts receives only one or a few.

We identify a trend in modern MoE models toward increasingly bimodal token-to-expert distributions, quantify the resulting disparity in arithmetic intensity across experts, and show that this disparity dramatically reduces the efficiency of state-of-the-art PIM systems for LLMs. To address this problem, we propose a scheduler for serving MoE models on multi-GPU systems with attached HBM-PIM stacks. Our scheduler partitions expert execution between GPU and PIM based on runtime token-to-expert distributions, while jointly considering interconnect overhead, memory bandwidth, GPU throughput, and PIM throughput. Moreover, we propose Sieve, a runtime framework that employs the scheduler to coordinate execution across GPUs and their attached HBM-PIM stacks. Sieve overlaps GPU computation, PIM computation, and intra- and inter-device communication while preserving cross-device dependencies induced by expert parallelism. Sieve is evaluated on our cycle-accurate simulator based on Ramulator 2.0. Compared to state-of-the-art PIM systems for MoE, Sieve improves both throughput and interactivity by 1.3×, 1.3×, and 1.6× on Qwen3.5-397B-A17B, GPT-OSS-120B, and Qwen3-30B-A3B, respectively.

Keywords

Mixture-of-Experts (MoE); Processing-in-memory (PIM); Inference Serving; Graphics Processing Unit (GPU); Dynamic Scheduling

1 Introduction

The Mixture-of-Experts (MoE) architecture has emerged as a leading direction for scaling LLM capacity efficiently [14, 15, 35, 47]. Instead of processing every token through a single dense feed-forward network (FFN), MoE models activate only a small subset of *experts* per token, dramatically increasing total parameter capacity without a proportional increase in per-token computation [14, 15, 35, 47]. State-of-the-art LLMs are increasingly adopting MoE layers to improve specialization and efficiency, and reduce training and inference cost, making MoE central to the next generation of LLMs [2, 10, 19, 39, 48, 49, 52].

At the same time, serving these increasingly large models is becoming bottlenecked not by arithmetic throughput, but by the cost of moving billions of parameters through the memory hierarchy [12]. This growing imbalance, known as the AI memory wall [17], has motivated the adoption of Processing-in-Memory (PIM) technology. PIM architectures embed lightweight compute units near DRAM banks to exploit the high internal bandwidth of modern memory devices [33, 34]. Prior work shows that PIM can substantially accelerate memory-bound components of LLM inference, such as attention layers [21, 22, 32, 44, 53].

However, modern MoE models are evolving in ways that fundamentally challenge existing PIM-enabled LLM systems. As illustrated in Figure 1, fewer experts are activated out of increasingly many [2, 39, 48, 49, 52], yielding a bimodal token-to-expert distribution where a small set of popular experts receives many tokens while a long tail of unpopular experts receives only one or a few. This distribution creates a large disparity in arithmetic intensity across experts. For example, in *Qwen3-Next* at batch size 64, 44.2%

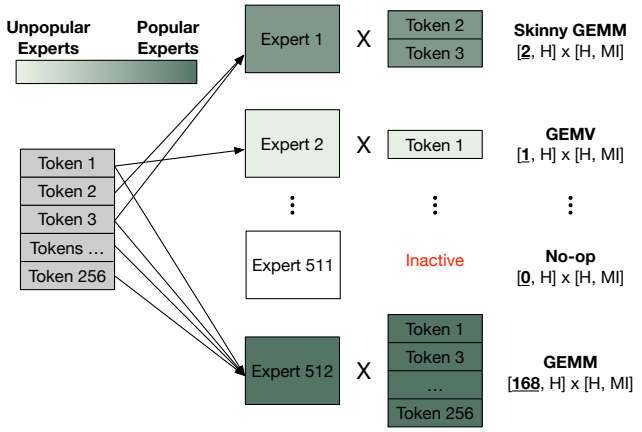

Figure 1: An example expert distribution in which token allocation varies across experts.

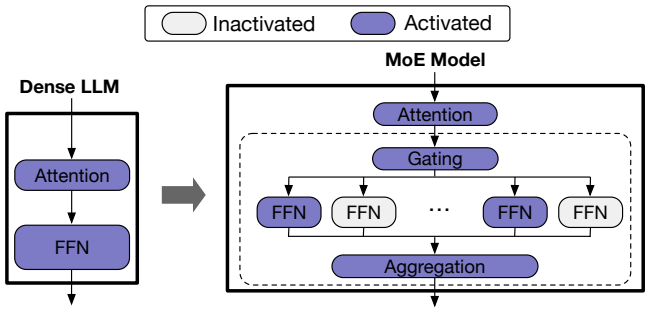

Figure 2: Comparison of the key differences between dense LLM and MoE architectures.

of experts receive only a single token, and 89.3% receive at most four, leaving only a small minority to process larger token batches.

We further show that this disparity makes state-of-the-art PIM systems inefficient. Existing PIM systems rely on **static** offloading rules, such as mapping only attention to PIM or using a fixed threshold to determine whether an expert should run on PIM [22, 32, 51]. Prior work also assumes a global interconnect across PIM devices, overlooking the combined effects of expert imbalance and inter-GPU communication that commonly arise when serving MoE models in multi-GPU systems [21, 22, 44, 51].

Building on this analysis, we propose a scheduler for serving MoE models on multi-GPU systems with HBM-PIM stacks. The scheduler exploits the arithmetic intensity disparity induced by the bimodal expert distribution to **dynamically** partition expert computation between GPUs and their attached HBM-PIM stacks. In general, the scheduler dispatches experts with low arithmetic intensity to PIM, and those with high arithmetic intensity to GPUs.

However, naively assigning memory-bound experts to PIM is suboptimal because expert parallelism across GPUs introduces inter-GPU communication, and the attention operations must co-execute with memory-bound experts on PIM. To improve the performance, our scheduler jointly accounts for interconnect overhead, memory bandwidth, GPU throughput, and PIM throughput. This differs from prior work, which neglects the all-to-all communication overhead and assumes that a GPU can access any PIM device [21, 22, 51]. Moreover, our scheduler uses a lightweight runtime algorithm, incurring only 20µs overhead on an NVIDIA B200 GPU.

We also design Sieve, a runtime framework that incorporates the new scheduler to enable practical deployment and efficient coordination of multi-GPU systems with HBM-PIM stacks. It coordinates execution across GPUs and their attached HBM-PIM stacks by overlapping PIM computation, GPU computation, and inter-GPU communication to maximize hardware utilization. In addition, Sieve employs expert parallelism across GPUs and tensor parallelism across the PIM channels attached to each GPU, thereby maximizing PIM utilization by ensuring that no PIM channel is left underutilized when memory-bound experts are assigned to PIM. It also preserves efficient GPU execution for popular experts through grouped GEMM, even when some tokens originally dispatched to the GPU are offloaded to PIM, requiring additional aggregation between the GPU and its attached PIM.

We evaluate Sieve using three state-of-the-art MoE models with different sparsity patterns, model sizes and ratios of activated to total parameters: GPT-OSS-120B (*GPT-OSS*), Qwen3.5-397B-A17B (*Qwen3.5*), and Qwen3-30B-A3B (*Qwen3*). Compared to the state-of-the-art PIM systems for MoE, Sieve achieves 1.3×, 1.3×, and 1.6× improved throughput and interactivity on *Qwen3.5*, *GPT-OSS*, and *Qwen3*, respectively.

In this paper, we make the following key contributions:

(1) We **identify and evaluate a trend in recent MoE models** toward increasingly bimodal expert distributions, which create disparities in arithmetic intensity across experts, and **show the impact of this trend** that renders state-of-the-art PIM-enabled systems inefficient.
(2) We propose a scheduler for serving MoE models in multi-GPU systems with PIM **that exploits the bimodal expert distribution and uses arithmetic intensity** derived from runtime token-expert distributions to dynamically partition expert computations between each GPU and its attached HBM-PIM stacks.
(3) We design Sieve, a runtime framework that employs the new scheduler to efficiently coordinate execution across GPUs and attached HBM-PIM stacks. Sieve **overlaps computation across GPUs and attached HBM-PIM stacks with both inter- and intra-device communication**.
(4) We evaluate Sieve on state-of-the-art MoE models and show that Sieve's gains are robust across batch sizes and workload distributions.

2 Background

The two key components of our work are Mixture-of-Experts (MoE) models and Processing-in-Memory (PIM) architectures. We first discuss the design of MoE architectures, and then review how PIM addresses the memory bottlenecks in LLM acceleration.

2.1 Mixture-of-Experts (MoE)

As illustrated in Figure 2, MoE models differ from dense LLMs by processing each token through a distinct combination of experts

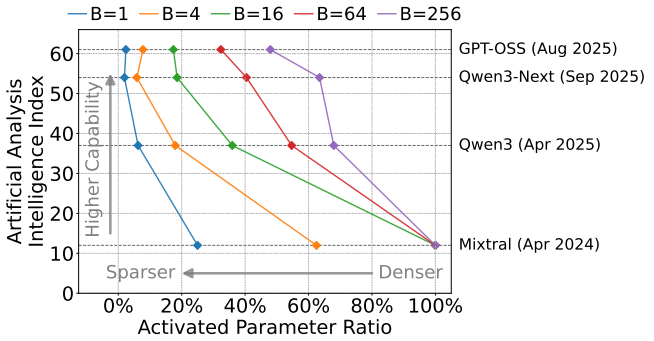

Figure 3: Benchmarking the sparsity–capability relationship. The plot shows the median activated-parameter ratio, where a lower ratio indicates a sparser model. AAII is considered batch-invariant.

instead of a single shared FFN. A MoE layer consists of multiple independent FFNs, referred to as "experts", and a gating network that determines which experts each token selects. The selected experts process their assigned tokens, and an aggregation unit combines the corresponding outputs. This selective activation allows MoE models to scale total parameter capacity dramatically while keeping per-token computation and memory requirements low.

2.2 Processing-in-Memory for LLMs

PIM has emerged as a promising architectural solution to the memory-wall problem in modern accelerators [20, 26, 27, 31, 33, 34]. By placing processing units (PUs) near DRAM banks, PIM architectures directly leverage the large internal memory bandwidth for computation, thereby bypassing the limited external memory bandwidth of GPUs. Since this internal bandwidth can exceed the external bandwidth by an order of magnitude, PIM provides strong potential for accelerating memory-bound operations. PIM architectures are particularly efficient for general matrix-vector multiplication (GEMV) operations, as their PUs are often specialized for dot product operations using adder trees [20, 27, 34]. To exploit bank-level parallelism, the vector operand is typically broadcast to all PUs within a PIM channel, while the matrix operand is partitioned across the banks associated with the PIM channel [20, 22, 32].

In the context of LLM inference, prior studies identify the attention operation in the decoding phase as memory-bound, characterized by low arithmetic intensity and dominated by GEMV operations [18, 21, 22, 32, 44, 53]. Based on this observation, PIM-enabled systems commonly offload attention operations to PIM, while compute-bound GEMM operations in FFN and QKV generation layers remain on GPUs or NPUs. This offloading strategy yields approximately a 2× speedup over PIM-disabled systems [18, 22, 32, 44, 53], indicating a shift from earlier accelerator designs optimized for GEMM-dominant machine learning workloads [8, 9, 25].

3 The Bimodal Expert Distribution Problem

In this section, we first explain the evolution of modern LLMs, particularly the shift toward increasingly sparse MoE architectures. Next, we identify how this shift creates a stark disparity in arithmetic intensity across experts and empirically quantify this disparity in recent MoE models. Finally, we demonstrate why this dynamic invalidates the static scheduling strategies used by existing PIM-enabled systems.

3.1 Recent LLM Trends

The progression from BERT [13] to GPT-3 [6] exemplifies how early LLM development pursued greater accuracy primarily through aggressive scaling of dense LLM architectures. However, MoE models decouple the growth in total parameter size from per-token computational cost by activating only a small subset of expert parameters for each token [14, 15, 35, 47]. Moreover, the ongoing evolution of MoE models reflects a clear trajectory toward greater sparsity, where models scale to a greater number of experts while activating a smaller fraction per token [2, 24, 41, 49].

Figure 3 visualizes this trend by comparing the activated parameter ratio with the Artificial Analysis Intelligence Index (AAII). The activated parameter ratio is calculated as the ratio of the activated parameter size to the total parameter size. AAII is a metric combining multiple dimensions of intelligence where a higher score indicates greater model capability [3]. Parameters in non-MoE layers are always activated and are therefore included in the activated parameter size. We measure the number of activated experts in MoE layers, which depends on input sequences, by running Mixtral-8x22B [24], Qwen3-30B-A3B [52], Qwen3-Next-80B-A3B [52], and GPT-OSS-120B [2] across various batch sizes (B) on traces of real-world requests [16]. For simplicity, we hereafter refer to these models as *Mixtral*, *Qwen3*, *Qwen3-Next*, and *GPT-OSS*, and denote the activated parameter ratio as *act-ratio*.

We draw two key observations from Figure 3.

> ***Observation 1:*** *Modern MoE models with higher capability exhibit lower act-ratios.*

Higher-capability MoE models consistently show lower activation ratios across batch sizes, as shown in Figure 3. For instance, at batch size $B = 1$, the act-ratio is 2.4% for *GPT-OSS* and 25.0% for Mixtral-8x22B. This trend has continued over the past few months: *Qwen3* (released in April 2025) shows higher act-ratios than *Qwen3-Next* (September 2025) and *GPT-OSS* (August 2025).

> ***Observation 2:*** *Modern MoE models exhibit low act-ratios even with larger batch sizes.*

Furthermore, even as batch size increases, modern MoE models continue to activate only a small fraction of their parameters, as shown in Figure 3. Although the act-ratio of the earlier *Mixtral* rapidly converges to 100%, that of *GPT-OSS* remains substantially lower. For example, the median act-ratio for *Mixtral* reaches 100% at $B = 16$, whereas *GPT-OSS* records 47.9% even at $B = 256$. *Qwen3-Next* also exhibits a similar trend to that of *GPT-OSS*.

In summary, Figure 3 demonstrates that the decreasing act-ratio represents a consistent trend among modern MoE models. This architectural shift towards greater sparsity fundamentally changes the execution characteristics of MoE serving, invalidating the static workload assumptions that prior PIM systems relied upon.

3.2 Arithmetic Intensity Disparity

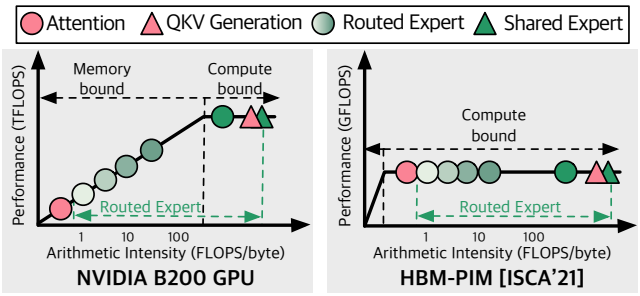

Figure 4: Roofline models of the NVIDIA B200 GPU [42] and the Samsung HBM-PIM [33]. Arithmetic intensities of operations in MoE models are illustrated, where the arithmetic intensity of each routed expert varies with its number of assigned tokens. Darker colors indicate experts with more routed tokens.

As described in Figure 1, the number of tokens assigned to each expert varies, creating disparities in arithmetic intensity across experts. To analyze how this disparity affects PIM-enabled systems, Figure 4 presents roofline models of the NVIDIA B200 GPU [42] and Samsung HBM-PIM [33]. The pink-colored attention and QKV generation operations have fixed arithmetic intensity determined by model configuration and sequence length, so their arithmetic intensities are known before inference begins. Since the attention operation is typically more memory-bound than other operations on GPUs, offloading the attention operation to PIM can yield substantial performance improvement [21, 22, 32, 44, 53].

In contrast, routed experts exhibit a broad range of arithmetic intensities, as shown by the large variation in arithmetic intensity among the green circles in Figure 4. Experts receiving only a few tokens have low arithmetic intensity and are inefficiently executed on GPU. On the other hand, experts with many tokens are compute-bound and achieve higher efficiency on GPU than on PIM. Consequently, assigning experts to PIM without considering their arithmetic intensity can lead to severe inefficiency.

To fully exploit PIM for efficient MoE serving, these disparities must be carefully addressed. Figure 4 provides a straightforward insight: experts with low arithmetic intensity are best executed on PIM, whereas those with high intensity should be processed on GPU. In the next subsection, we quantify the degree of disparity in arithmetic intensity across experts to analyze its performance implications for PIM-enabled systems.

3.3 Quantifying Arithmetic Disparity

We draw two key observations from Figure 5, which illustrates the degree of disparity in arithmetic intensity across experts.

> ***Observation 3:*** *A substantial fraction of expert computations remain memory-bound even with large batch sizes.*

The number (N) of tokens assigned to each expert determines its arithmetic intensity because all experts within a MoE layer share identical parameter tensor dimensions [2, 10, 19, 39, 48, 49, 52]. Accordingly, we classify experts into two groups: (1) memory-bound (unpopular) experts performing GEMV or skinny GEMM operations, and (2) compute-bound (popular) experts performing GEMM operations. We bin experts into $N = 1$, $N = 2$, $3 \leq N \leq 4$, and $N > 4$ to characterize arithmetic intensity in Figure 5. For shared experts in MoE models such as *Qwen3-Next* [52], N equals the batch size (B), typically resulting in compute-bound operations when $B > 4$.

As expected, memory-bound expert computations dominate at small batch sizes. For example, when $B = 4$, 92.5% of expert computations in *Qwen3-Next* correspond to GEMV. Moreover, a significant portion of expert computations in modern MoE models remains memory-bound even at larger batch sizes. The earlier MoE model *Mixtral* shows almost no memory-bound experts once $B \geq 64$. On the other hand, as we move to newer MoE models, the ratio of memory-bound expert computations grows notably. Newer-generation MoE models show a drastic difference: even at $B = 64$, 47.6% of expert computations in *Qwen3*, 89.3% in *Qwen3-Next* and 65.9% in *GPT-OSS* are memory-bound. This high ratio persists even when $B = 256$, where 50.1% and 56.6% of experts in the respective models remain memory-bound. Moreover, a striking finding from Figure 5 is that a large portion of expert computations continues to be memory-bound even in large-batch scenarios for recent models, which are designed to improve throughput.

> ***Observation 4:*** *Single-token assignments (GEMV) occur frequently, indicating a high proportion of low arithmetic intensity computations.*

The difference in single-token assignments across generations of MoE models is particularly noteworthy, as these computations degenerate into GEMV operations. We refer to experts that are assigned exactly one token in a batch as *GEMV experts*, since computation in the experts is composed of GEMV operations. At $B = 64$, GEMV experts account for 20.2% of expert computations in *Qwen3*, 32.6% in *GPT-OSS*, and 44.2% in *Qwen3-Next*. This trend persists even at $B = 256$, where the corresponding ratios are 11.9%, 23.5%, and 23.9%, respectively. Although Figure 5 extends to $B = 1024$, batch sizes below 256 are generally preferred to avoid excessive latency and potential SLO violations [21, 22, 32, 36, 44, 53].

In summary, the large disparity in arithmetic intensity across experts calls for an adaptive acceleration approach, which our framework is designed to provide.

3.4 Limitations of Prior PIM-enabled Systems

Prior PIM-enabled systems mainly target memory-bound operations in dense LLMs [18, 21, 22, 32, 44, 53]. As discussed in Section 2.2, these systems commonly offload attention operations during the decoding phases to PIM, because attention has low arithmetic intensity and benefits from the high internal bandwidth of PIM. In contrast, compute-intensive operations such as FFNs and QKV generation remain on the GPU. To overlap computations between GPU and PIM, most prior work splits each batch (N) into two mini-batches ($N/2$) and interleaves their execution across the two devices, as illustrated in Figure 6 (a). This interleaving reduces the idle time of both PIM and GPU when running dense LLMs. A

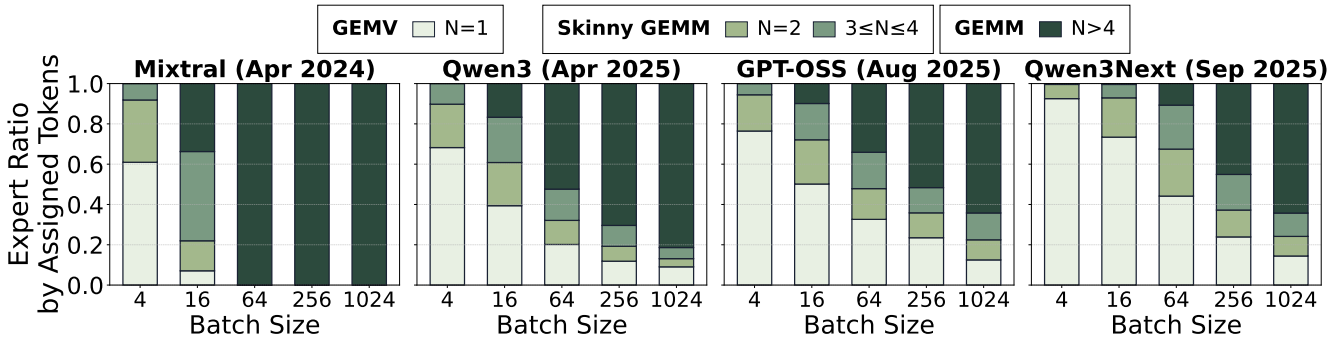

Figure 5: Proportion of GEMV, skinny GEMM, and GEMM experts in *Mixtral*, *Qwen3*, *GPT-OSS*, and *Qwen3-Next*, averaged over the HH-RLHF dataset [16]. N denotes the number of tokens assigned to each expert. For example, in *Qwen3-Next*, the first FFN has a weight matrix of shape $[2048, 512]$, and its computation can be represented as a matrix multiplication of dimensions $[N, 2048] \times [2048, 512]$. The shared expert in *Qwen3-Next* is also included. These bins are used only to expose arithmetic disparity; they are not Sieve scheduling thresholds.

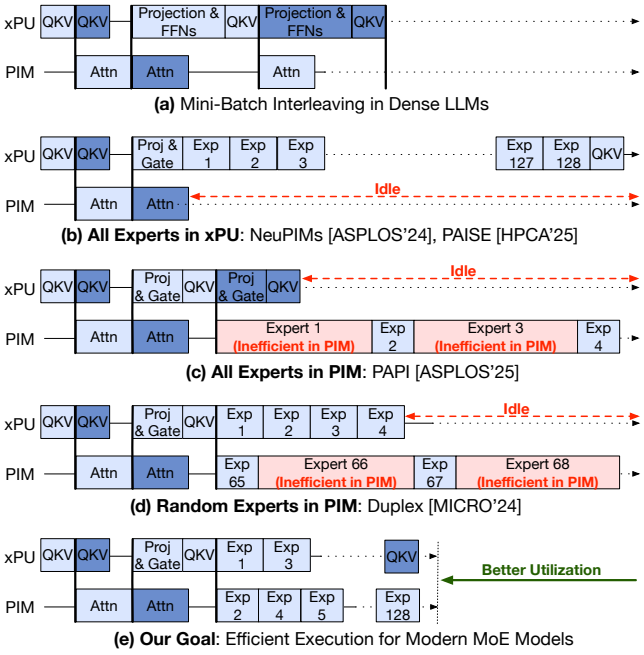

Figure 6: Execution flow examples in PIM-enabled systems for dense LLMs and MoE models. We use xPU as a generalized term for the host processor because prior work targets different compute substrates, including GPUs and NPUs.

naive sequential execution for dense LLMs in systems composed of GPUs and PIM would offload the attention operation to PIM. The mini-batch interleaving strategy utilizes the fact that dependencies exist only within a mini-batch, and it improves the throughput of PIM-enabled systems by reducing idle times. Furthermore, since serialized execution limits throughput by leaving one device idle, prior work interleaves two mini-batches, executing one on the GPU and the other on the PIM concurrently, as described in Figure 6 (a) [22, 53].

However, these strategies become inefficient when applied to modern MoE models. As illustrated in Figure 6 (b), PIM remains idle during expert computation in systems that offload only attention operations to PIM [22, 32]. Although GPUs can process multiple experts using batched matrix multiplication instead of executing each expert sequentially, the overhead of loading all activated expert parameters from off-chip memory remains substantial, preventing full PIM utilization. Similarly, as shown in Figure 6 (c), offloading all expert computations to PIM causes the GPU to remain idle [21]. Therefore, to fully exploit PIM-enabled systems, the expert layer should be co-processed across both PIM and GPU.

As described in Figure 6 (d), naively co-processing FFNs on both PIM and GPU also introduces inefficiency in modern MoE models. Prior work evaluates such co-processing by assuming a uniform expert distribution [53], which was valid for earlier MoE models such as *Mixtral* with large batch sizes. In contrast, as elaborated in Section 3.3, modern MoE models exhibit highly imbalanced token-to-expert assignments even with large batches. Executing experts with many assigned tokens on PIM becomes inefficient because commercial PIM architectures are optimized for GEMV rather than GEMM, resulting in lower throughput.

In summary, existing PIM strategies designed for dense LLMs and earlier MoE models are ineffective for modern MoE models. Figure 6 (e) illustrates the execution flow that this work aims to achieve, where MoE serving is optimized to better utilize both PIM and GPU resources for higher throughput.

4 Overview

Building on the aforementioned insights, we introduce Sieve, a dynamic expert-aware PIM acceleration framework for serving MoE models. Sieve leverages the disparity in arithmetic intensity across experts and the complementary characteristics of GPUs and PIM. Sieve requires no hardware modifications: no new PIM commands and no changes to the existing PIM architecture or command interface. Therefore, Sieve incurs no additional hardware cost, such as area overhead. Instead, Sieve improves the utilization

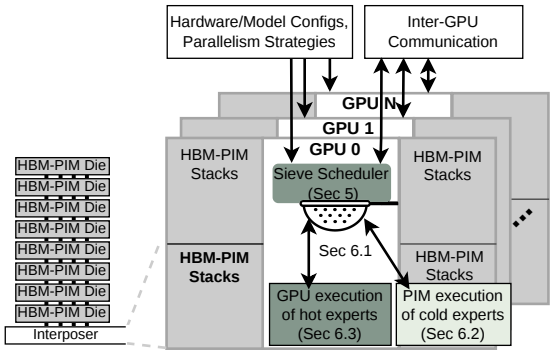

Figure 7: An overview of Sieve. Hardware and model configurations are determined before serving MoE models. The Sieve scheduler leverages the runtime-determined distribution of tokens across experts to enable efficient co-execution of GPU and PIM computations.

of existing PIM resources by exploiting the interaction between PIM architectural characteristics and modern MoE serving workloads.

Memory Model: A PIM die is a conventional HBM die augmented with PIM processing units, trading a portion of memory capacity for compute capability while maintaining approximately the same die area as a standard HBM die. Expert parameters are stored in off-chip HBM-PIM dies across GPUs, depending on the expert-parallelism and tensor-parallelism configuration. Expert migration across GPUs may arise when expert-parallel load balancing (EPLB) is enabled, which can be utilized with Sieve. For GPU execution, expert parameters are fetched from off-chip memory into GPU on-chip memory before computation. For PIM execution, PIM processing units directly access expert parameters from local DRAM banks, without transferring them to GPU on-chip memory. Accordingly, Sieve determines *that each expert is executed on either PIM or GPU*, rather than where its parameters are stored.

Interconnect Model: Prior work assumes a global interconnect across multiple GPUs (or NPUs) and PIM devices [21, 22, 44, 51]. Under this assumption, any GPU can access any PIM device. This abstraction fails to capture inter-GPU communication overhead caused by expert parallelism across GPUs, even though such overhead is a key bottleneck in MoE inference.

We instead assume that each GPU integrates multiple HBM-PIM stacks, where a stack consists of multiple HBM-PIM dies as described in Figure 7. Each HBM-PIM die is accessible only through its attached GPU. Therefore, in this setting, a token cannot access an expert in a remote HBM-PIM stack without going through the corresponding remote GPU. This design matches real-world multi-GPU systems, where GPUs are interconnected via NVLink but HBM dies are not directly connected to other GPUs.

Sieve System: Executing all experts on GPUs incurs significant data movement overhead for transferring all activated experts' parameters from HBM to GPU before computation. In contrast, executing all experts on PIM eliminates this transfer cost but reduces overall performance due to PIM's lower computational throughput compared to GPUs. A static assignment of experts to PIM is also inefficient, as the number of tokens assigned to each expert varies dynamically across batches at runtime. Therefore, efficient execution of modern MoE models requires adaptively partitioning expert workload between GPUs and PIM at runtime.

Sieve adopts a dynamic approach that partitions experts into two groups based on their arithmetic intensity. Memory-bound experts with low arithmetic intensity are executed on PIM, while compute-bound experts with high intensity are executed on GPUs. By executing certain experts on PIM, Sieve reduces off-chip memory access overhead. Moreover, Sieve dynamically decides the partition while accounting for the overhead of inter-GPU communication and the execution of attention operations on PIM.

Figure 7 shows an overview of Sieve. Based on the runtime token-to-expert distribution and the overhead of inter-GPU communication and PIM-based attention operations, the Sieve scheduler (Section 5) efficiently partitions expert execution between GPUs and PIM. Section 6 then presents the Sieve system, which realizes this partitioning through three key components: lightweight coordination across GPUs and their attached HBM-PIM stacks (Section 6.1), efficient execution of unpopular experts on PIM (Section 6.2), and efficient execution of popular experts on GPUs (Section 6.3).

5 Sieve Scheduler

The Sieve scheduler is a runtime scheduler for MoE models. It dynamically partitions expert computations between GPUs and their attached HBM-PIM stacks. This subsection describes how the Sieve scheduler determines an effective scheduling strategy, accounting for factors such as the MoE architecture, hardware configuration, and parallelization strategies, including data, context, expert, and tensor parallelism.

5.1 Objective Function

The Sieve scheduler leverages the arithmetic intensity disparity caused by the bimodal token–expert distribution at runtime to make efficient scheduling decisions. To maximize throughput and minimize latency, the Sieve scheduler follows three core principles: ① It jointly considers hardware utilization across four resources: the interconnect, PIM, GPU compute, and GPU off-chip memory bandwidth. ② Popular experts are executed on GPUs to avoid performance degradation. ③ Unpopular experts with low arithmetic intensity are offloaded to PIM. Its decisions are guided by the runtime expert distribution, hardware configuration, and MoE model configuration.

Principle ① defines the primary objective of the Sieve scheduler, while ② and ③ specify how the Sieve scheduler achieves it. We formalize principle ① with the objective function in Equation 1. Let E denote the set of activated experts on a GPU and its corresponding HBM-PIM stacks, $S \subseteq E$ the subset assigned to PIM, and $G = E - S$ the subset assigned to the GPU. The Sieve scheduler finds the partition S^* that minimizes the bottleneck across three components:

$$S^* = \arg\min_{S \subseteq E} \max\Big(T_{\text{Comm}},\ T_{\text{GPU}}(G),\ T_{\text{PIM}}(S)\Big) \tag{1}$$

Prior work has proposed highly fine-grained overlap of computation and communication to maximize GPU utilization during MoE inference [54, 57]. Accordingly, the Sieve scheduler uses the

maximum estimated execution time across the interconnect, GPU, and PIM. Since the Sieve scheduler runs on the critical path, it is designed to identify the dominant bottleneck with low overhead rather than exactly predict execution time. The detailed execution times reported in Section 7.2 are obtained from cycle-accurate simulation.

T_{Comm} denotes the estimated inter-GPU communication time incurred by tensor and expert parallelism. Because tokens are routed to other GPUs totally based on the gating results regardless of the Sieve scheduler's partitioning decision, T_{Comm} is independent of both S and G. $T_{\text{GPU}}(G) = \max\left(T_{\text{offchip}}(G), T_{\text{comp}}(G)\right)$ denotes the estimated execution time for GPU operations, such as popular experts retained on the GPU by principle ②. The Sieve scheduler determines it as the larger of $T_{\text{offchip}}(G)$ (the off-chip memory access time for loading parameters and storing intermediate values) and $T_{\text{comp}}(G)$ (the GPU computation time). $T_{\text{PIM}}(S)$ denotes the estimated execution time for PIM operations, such as attention and unpopular experts assigned to PIM by principle ③.

Timing Models: The following estimates for $T_{\text{GPU}}(G)$, $T_{\text{PIM}}(S)$, and T_{Comm} are used solely within the Sieve scheduler to guide the partitioning decision, while the execution times reported in Section 7.2 are obtained from cycle-accurate simulation. Since the Sieving algorithm runs on the critical path, we prioritize lightweight estimates over precise modeling to keep its overhead low.

$T_{\text{offchip}}(G)$ is computed by dividing the amount of data transferred between HBM-PIM and the GPU, including parameters and intermediate values, by the HBM bandwidth. $T_{\text{comp}}(G)$ is estimated by dividing the number of GPU operations, including all operations except decode-phase attention and PIM-side expert computation (S), by the GPU's peak compute throughput. Specifically, the Sieve scheduler assumes that GPU-side expert computation is efficiently executed using grouped GEMM [30, 55], where the experts in G are batched into a single kernel with variable group sizes determined by token counts. T_{Comm} is estimated by dividing the amount of inter-GPU communication incurred by tensor and expert parallelism by the interconnect bandwidth.

For $T_{\text{PIM}}(S)$, we follow the assumption in prior work that PIM executes a multi-token expert as serialized GEMV operations through the DRAM command interface [22, 33]. However, prior work has shown that these overheads are non-linear [32]. Executing a 1-token expert on PIM does not take half the time of executing a 2-token expert, mainly because of DRAM timing overheads such as row buffer conflicts, bank contention, and refresh cycles. As a result, a roofline-based estimate can overestimate expert execution time on PIM by 1.8–4.2×, because it does not capture DRAM timing overhead.

To address this, the Sieve scheduler maintains a runtime cost table whose keys are token counts and whose values are the observed PIM execution times for experts with those token counts. After each iteration, the Sieve scheduler updates the cost table using an exponential moving average of the observed PIM GEMV execution times. For token counts that have not yet been observed, the Sieve scheduler uses a roofline estimate obtained by dividing the number of operations by the PIM's peak compute throughput. Although this estimate may be inaccurate, the scheduler uses it only as a one-time fallback until an observed PIM timing estimate becomes available. The PIM cost table converges within the first few iterations, as the varying expert distributions across successive batches quickly populate entries for the relevant token counts.

5.2 Sieve Scheduling Algorithm

As defined in Equation 1, the Sieve scheduler seeks to identify an optimal $S^* \subseteq E$ such that the estimated execution time is minimized. However, an exhaustive search across all $2^{|E|}$ combinations is computationally infeasible; for instance, Qwen3-Next-80B-A3B would require evaluating 2^{512} combinations.

To find S^* efficiently, the Sieve scheduler employs a greedy heuristic. First, it sorts all experts in descending order based on their token counts. The Sieve scheduler assumes all expert computations are initially assigned to PIM. It then iteratively evaluates whether moving the computation of the expert with the highest token count from PIM to the GPU would reduce the estimated overall execution time $T_{\text{total}} = \max\Big(T_{\text{Comm}},\ T_{\text{GPU}}(G),\ T_{\text{PIM}}(S)\Big)$. If T_{total} decreases, the expert computation is reassigned to the GPU. Assigning the most popular expert to the GPU yields the largest reduction in T_{PIM}, while the GPU cost of executing an additional expert is relatively constant as most experts are memory-bound on the GPU. The Sieve scheduler continues until moving the next expert increases T_{total}, indicating that the remaining experts have sufficiently low arithmetic intensity to be better handled by PIM. This process also stops if all expert computations have been assigned to the GPU, which is a common outcome in MoE models with a small number of experts such as Mixtral-8x7B.

Overhead of the Sieve Scheduler: The Sieve scheduler is designed to be lightweight and practical for modern MoE models where $|E| < 1024$. By prioritizing the most popular experts, the scheduler achieves the most significant reductions in T_{PIM} with minimal iterations. Since the complexity is dominated by a single sort and a linear scan over the expert list, the computational overhead of the Sieve scheduler is negligible. For example, the Sieve scheduler runs in approximately 20μs on a B200 GPU for a MoE layer of DeepSeek-R1, even without kernel optimization. In our evaluation, each GPU executes this algorithm locally after receiving global token counts via the AllGather step explained in Section 6.1, and this overhead is fully accounted for in our results.

Comparison with PIMoE [51]: PIMoE uses a static *threshold* to assign popular experts with $N \geq$ *threshold* to the NPU when the estimated NPU execution time is lower than the estimated PIM execution time. However, PIMoE partitions expert computations without accounting for attention operations that already occupy PIM. As attention time on PIM grows with longer sequences or higher request concurrency, a partition that appears balanced for an MoE operation can become inefficient in end-to-end MoE inference. This is because the PIM execution time increases substantially, turning PIM into the bottleneck.

PIMoE also assumes a global interconnect that allows an NPU to access any PIM device with uniform latency. This assumption ignores a key overhead in real-world multi-GPU systems: the inter-GPU communication required for token dispatch and combination in expert parallelism. When PIMoE is adapted to multi-GPU systems in which each GPU integrates multiple HBM-PIM stacks, the PIM execution cost exceeds both the inter-GPU communication

cost and the GPU execution cost. In this case, assigning more experts to the GPU can reduce PIM execution time and increase GPU utilization. Therefore, rather than relying on a static threshold, the Sieve scheduler must dynamically consider both inter-GPU communication and attention time on PIM.

6 Sieve System

This section describes the Sieve runtime system, which turns the Sieve scheduler's per-expert placement decisions into an executable MoE-layer pipeline. The runtime must coordinate routing metadata, token dispatch, GPU/PIM execution, and final aggregation across GPUs and their attached HBM-PIM stacks. We first describe this cross-device coordination in Section 6.1, then discuss the PIM and GPU execution paths in Section 6.2 and Section 6.3.

6.1 Coordinating GPUs and HBM-PIM

Sieve coordinates each MoE layer as a dependency graph over routing, communication, and expert execution. This graph exposes independent operations that can be overlapped across inter-GPU communication, GPU compute, and HBM-PIM compute, while preserving the ordering required for correctness.

Dependencies across operations: An example dependency graph for co-executing MoE layers on a PIM-enabled multi-GPU system is shown in the left panel of Figure 8. From the attention output (❶), the router (❷) computes a token-to-expert routing map. With data-parallel attention, each GPU initially holds only a local routing map containing a disjoint subset of tokens. Since expert parallelism is used across GPUs, an AllGather step (❸) aggregates these local maps into a global view. This allows each GPU to identify where the tokens assigned to each expert reside. During a metadata processing step (❹), each GPU prepares fixed-size tensors per expert, enabling efficient grouped GEMM execution on the GPU.

After this stage, Sieve enables parallel co-execution on GPUs and PIM to efficiently utilize multiple resources: inter-GPU bandwidth, GPU compute, PIM compute, and HBM-PIM bandwidth. First, tokens are dispatched across GPUs (❺$_{\text{Dispatch}}$) so that each token resides on the GPU hosting its assigned expert. In parallel, the Sieve scheduler explained in Section 5 runs on each GPU (❺$_{\text{Sieve}}$) to determine which experts should execute on the GPU versus PIM.

If the Sieve scheduler assigns an expert to the GPU, its parameters are loaded from HBM-PIM to GPU (❻$_{\text{HBM-PIM}\rightarrow\text{GPU}}$). Since shared experts [2, 11, 19, 52] receive every token and thus lead to a large GEMM, we start loading the weights for these shared experts right after ❹. This enables more overlap by relaxing the dependency ❹→❺$_{\text{Dispatch}}$ →❻$_{\text{weight}}$ for shared experts. As an optimization, parameter loading can also begin early for experts assigned multiple tokens within the local routing map. Once loaded, these experts are processed via grouped GEMM on the GPU (❼$_{\text{GPU}}$). If Sieve assigns an expert to PIM, the GPU issues PIM commands to HBM-PIM while sending the tokens (❻$_{\text{GPU}\rightarrow\text{HBM-PIM}}$). The expert FFN is then executed directly on PIM (❼$_{\text{HBM-PIM}}$) and the produced tokens get loaded back to the GPU (❽).

To summarize, the popular experts are executed on the GPU through ❶→❷→❸→❹→❺$_{\text{Sieve}}$→❻$_{\text{HBM-PIM}\rightarrow\text{GPU}}$→❼$_{\text{GPU}}$→ ❾ with an additional dependency (❺$_{\text{Dispatch}}$→❼$_{\text{GPU}}$) for tokens dispatched from remote GPUs. The unpopular experts are executed on PIM through ❶→❷→❸→❹→❺$_{\text{Sieve}}$→❻$_{\text{GPU}\rightarrow\text{HBM-PIM}}$→❼$_{\text{HBM-PIM}}$ →❽→❾ with an additional dependency (❺$_{\text{Dispatch}}$→❻$_{\text{GPU}\rightarrow\text{HBM-PIM}}$) for tokens dispatched from remote GPUs.

Synchronization: Since Sieve executes unpopular experts by converting each of them into a GEMV operation, the number of GEMV operations on PIM can vary across batches. However, the dimensionality of each GEMV remains constant for a given MoE model. This property allows the memory controller to schedule PIM commands deterministically [22]. As a result, the sequence ❻$_{\text{GPU}\rightarrow\text{HBM-PIM}}$→❼$_{\text{HBM-PIM}}$→❽ proceeds without violating DRAM timing parameters such as refresh intervals. Moreover, to ensure that ❾ starts only after both ❼$_{\text{HBM-PIM}}$ and ❼$_{\text{GPU}}$ have completed, Sieve encodes these data dependencies within its DAG representation, following prior work [22].

Aggregation: Depending on the parallelization strategy in a system, additional aggregation between GPU and HBM-PIM may be required if some tokens dispatched to a GPU are computed on its corresponding HBM-PIM. Each dispatched token and the result of its expert computation have dedicated on-chip memory addresses after ❺$_{\text{Dispatch}}$. Therefore, although the token's expert computation is conducted in PIM and its result is loaded back to the GPU (❻$_{\text{GPU}\rightarrow\text{HBM-PIM}}$→❼$_{\text{HBM-PIM}}$→❽), the result can be stored again at the dedicated on-chip memory address. After that, ❾ is performed to send the dispatched token to the original GPU, where the aggregation of the token's intermediate values from multiple expert computations is executed.

6.2 Executing Unpopular Experts on PIM

After the Sieve scheduler decides the expert computation orchestration, PIM performs computation for GEMV and skinny GEMM experts. Sieve introduces a PIM-friendly MoE execution model for these experts.

Issuing PIM Commands: To support heterogeneous execution between GPUs and PIM, Sieve employs a custom GPU kernel that initializes and controls PIM operations. This kernel issues PIM commands using tensor sizes and memory addresses determined dynamically at runtime. As a result, Sieve can be realized entirely in software on GPUs. This design is feasible because prior studies have extended GPU programming models to allow custom kernels to manage PIM operations [22, 28, 53].

As shown in Figure 8, expert computation on PIM involves three sub-steps: (i) distributing the token tensor to the global buffers of all PIM channels and activating the corresponding rows of the operand matrix in the row buffers for PIM computation (❻$_{\text{token}}$), (ii) issuing the GEMV operation (❻$_{\text{cmd}}$) to perform expert FFN computation via a series of dot products (❼$_{\text{HBM-PIM}\rightarrow\text{GPU}}$), and (iii) reading the GEMV results back from PIM to GPU on-chip memory (❽$_{\text{token}}$). Sieve can be implemented on any PIM architecture or command interface that supports these three sub-steps. For example, in NeuPIMs [22], substep (i) is implemented with the *PIM_GWRITE* command, and substeps (ii) and (iii) are implemented with the *PIM_GEMV* command.

When the custom GPU kernel triggers PIM execution, it issues PIM commands whose arguments are computed based on the output of the Sieve scheduler. Since unpopular experts are executed through separate PIM commands, the arguments differ across GEMV operations. For example, the arguments include the row and

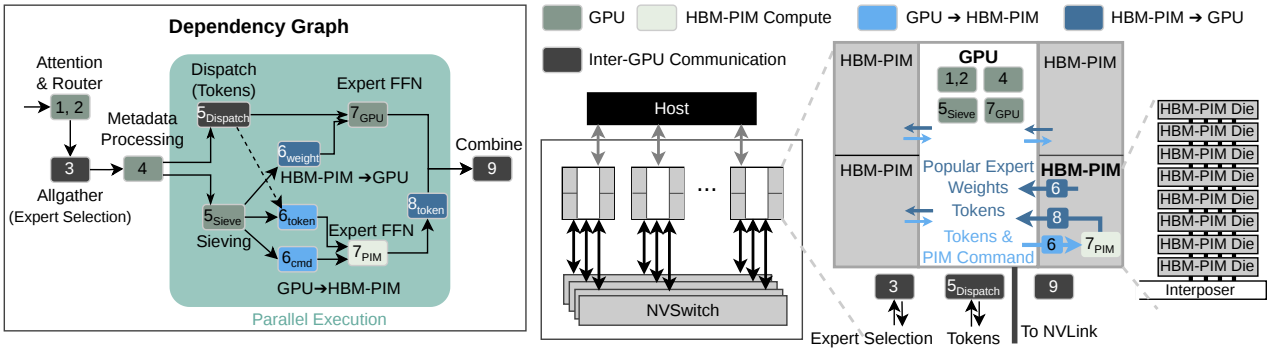

Figure 8: An overview of the Sieve System and the dependencies across operations.

column indices of the memory array that stores expert parameters for GEMV computation in sub-step (ii), and the GPU on-chip memory address used to load the results back in sub-step (iii). These addresses vary across PIM channels because each channel produces results for a different portion of the expert output. Preparing these arguments requires only basic arithmetic operations, which can be performed on the GPU at runtime, as discussed in prior work [22, 28, 53].

Converting Skinny GEMM to GEMV Operations: Commercial PIM designs are optimized for dot-product operations rather than GEMM operations [20, 27, 34]. To align with these architectures, Sieve converts each skinny GEMM expert into a sequence of equivalent GEMV operations and issues the corresponding PIM commands. For example, if an expert is selected by three tokens, Sieve performs three GEMV operations on PIM. In this case, substeps (i), (ii), and (iii) are executed sequentially on PIM for each token associated with that expert.

Parallelizing Expert Computation on PIM: Sieve adopts tensor parallelism across PIM channels to maintain high utilization even when the expert distribution is highly imbalanced. In other words, each expert's parameters are evenly sharded across all PIM channels, allowing every GEMV operation to be divided across the channels. Moreover, the parameters of a given expert are aligned to identical indices across banks and channels, which reduces the address calculation overhead. As a result, all PIM resources can be efficiently utilized even under dynamically changing and imbalanced expert distributions. Since this form of parallelism uses existing PIM commands, it remains compatible with current DRAM-based PIM interfaces used in prior work [22, 44].

An alternative is expert parallelism (EP), which executes different experts concurrently on separate hardware components because expert computations are independent [7, 30, 55]. In PIM architectures, EP can be applied at multiple granularities, such as banks, bank groups, and channels. However, assigning individual banks or bank groups to distinct experts is inefficient because all processing units within a PIM channel must share the same vector operand during each GEMV operation [20, 22, 32, 33]. Banks or bank groups without activated experts for a given token remain idle, leading to low utilization. Although channel-level EP can leverage inter-bank parallelism, it also risks low utilization when some channels contain experts selected by very few tokens, as discussed in Section 3.2. Supporting cross-channel or cross-bank access would require dedicated hardware mechanisms that are rarely available in commercial PIM systems [4]. Therefore, Sieve does not adopt EP because it cannot ensure high and balanced PIM utilization.

Prior work has also explored distributing the attention operation and KV cache of each request across PIM channels [22, 53]. However, distributing expert computation in the same manner is inefficient. Tokens from different requests may require the same expert parameters, which demands cross-channel access or redundant copies of expert parameters in multiple channels. Consequently, offloading expert computations to PIM requires a parallelization strategy that differs from attention.

6.3 Executing Popular Experts on GPUs

After running the Sieve scheduler described in Section 5, Sieve identifies the popular experts and their assigned tokens. To execute these GEMM experts on GPUs and store their intermediate results in GPU on-chip memory, Sieve follows the common practice in state-of-the-art LLM serving frameworks [7, 30, 50, 55]. At ❼GPU in Figure 8, Sieve performs the computation for popular experts on GPUs using grouped GEMM or batch matrix multiplication. The outputs of all popular experts are written into a contiguous region of GPU on-chip memory. After the outputs of all unpopular experts have also been transferred from PIM to GPU on-chip memory, Sieve reorders the expert-grouped results into token-grouped results. This permutation enables the final aggregation that computes the weighted sum for each token, yielding the MoE layer output.

7 Evaluation

7.1 Methodology

Simulation Methodology: We develop a cycle-accurate simulator using Ramulator 2.0 [40] and Duplex [53] to evaluate the performance of Sieve. Table 1 summarizes the GPU and HBM-PIM configurations used for detailed simulations of DRAM and PIM commands, following prior work [22, 28, 53]. Our PIM implementation includes state-of-the-art PIM hardware components for LLMs, such as dual row buffers in NeuPIMs [22]. Our cycle-accurate DRAM

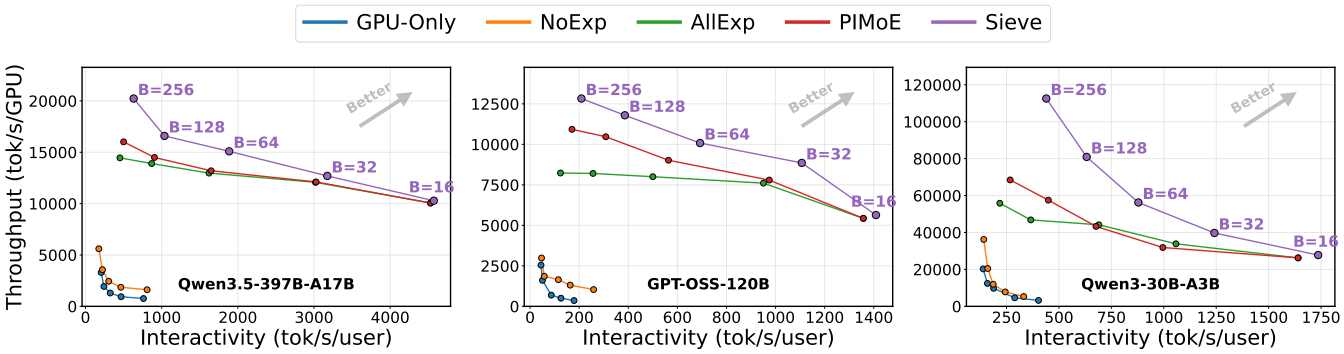

Figure 9: Evaluation of throughput and interactivity achieved by *GPU-Only*, *NoExp* [22, 32], *AllExp* [21, 43], *PIMoE* [51], and Sieve. *NoExp* and *AllExp* execute all expert computation on GPUs and PIM, respectively. *PIMoE* uses a static threshold to assign expert computation to GPUs and their attached HBM-PIM stacks. Four B200 GPUs, eight B200 GPUs, and one B200 GPU are used for *GPT-OSS*, *Qwen3.5*, and *Qwen3*, respectively, where each GPU has its own HBM-PIM stacks.

GPU Configuration (B200 GPU)		HBM-PIM Configuration	
FP16 throughput	2,250 TFLOPS	HBM generation	HBM3E
HBM-PIM bandwidth	8.0 TB/s	Pin rate	8.0 Gbps
HBM-PIM stacks	8	Pseudo-channels/stack	32
HBM-PIM capacity†	96 GB	Banks/pseudo-channel	24
NVLink BW (per dir.)	900 GB/s	Page size	1 KB
NVLink latency	0.8 μs	Compute density	1 op/byte
HBM3E Timing Parameters (cycles @ 8.0 Gbps, tCK ≈ 0.50 ns)			
tRCD = 28, tRP = 28, tRAS = 68, tRC = 96, tCL = 28, tWR = 32, tCCD_S = 2, tCCD_L = 4, tRRD_S = 6, tRRD_L = 6, tFAW = 12, tREFI = 3,900 ns, tRFC = 400 ns			

Table 1: Specification of the evaluated B200 multi-GPU system with HBM-PIM. †PIM processing units sacrifice 50% of HBM capacity (192→96 GB per GPU).

simulation via Ramulator 2.0 ensures that the control-path throughput is accurately measured and that no timing violations occur between DRAM and PIM commands [33]. We adopt a performance model for GPUs from Duplex [53], assuming experts are executed via grouped GEMM on GPUs. We configure the GPU to match the NVIDIA B200 GPU as shown in Table 1. All experiments assume multi-GPU systems with HBM-PIM stacks where each HBM-PIM die is accessible only through its attached GPU. Although Sieve is evaluated by simulating multi-GPU systems with HBM-PIM stacks in which each HBM-PIM die is accessible only through its attached GPU, its core ideas also apply to the global interconnect scenarios considered in prior work, where any XPU in the system can directly access any PIM device [21, 22, 44, 51].

Models: We evaluate Sieve on GPT-OSS-120B (*GPT-OSS*), Qwen3.5-397B-A17B (*Qwen3.5*), and Qwen3-30B-A3B (*Qwen3*). These models also represent the recent LLM trends discussed in Section 3.1. *GPT-OSS* and *Qwen3* activate four experts out of 128, while *Qwen3.5* activates ten experts out of 512 with one shared expert. Furthermore, *Qwen3.5* and *GPT-OSS* reflect the trend toward greater sparsity in state-of-the-art MoE models, but along different dimensions. Earlier MoE models typically activate eight experts out of 128 or 256 total experts [5, 19, 39, 52]. *Qwen3.5* increases the act-ratio by increasing the total number of experts, whereas *GPT-OSS* does so by lowering the ratio of activated to total experts. We evaluate a range of batch sizes (B) up to 256 to satisfy common Service Level Objectives (SLOs) [1, 21, 22, 28, 32, 36, 53]. We use four B200 GPUs for *GPT-OSS*, eight B200 GPUs for *Qwen3.5*, and a single B200 GPU for *Qwen3*, where all GPUs are attached to their HBM-PIM stacks.

Datasets: We collect real expert distributions by running the MoE models on GPUs, using real-world request traces from the HH-RLHF [16] and MATH-500 [38] datasets. HH-RLHF contains human preference comparisons for long, open-ended dialogue sequences across diverse topics [16]. MATH-500 is a benchmark of competition-style math problems with formal statements and proof-like reasoning [38]. Unless stated otherwise, activations and parameters are stored in bfloat16.

Comparison Methodology: We compare Sieve against three methods that statically determine which experts are executed on PIM: *NoExp*, *AllExp*, and *PIMoE*, each reproducing a state-of-the-art in- and near-memory computing technique for LLM inference [21, 22, 32, 43, 44, 51, 53]. Each method incorporates the same non-MoE optimizations, such as offloading attention operations to PIM [21, 22, 32, 44, 53], distributing KV cache across PIM channels [22], and maintaining separate paths for DRAM and PIM commands [22, 53]. All methods share identical hardware configurations and differ only in how expert computations are scheduled between GPUs and their HBM-PIM stacks.

NoExp denotes a method where only attention operations are offloaded to PIM and all experts are executed on GPUs [22, 32]. *NoExp* with attention operations offloaded to PIM is the dominant and widely studied PIM acceleration strategy, providing a consistent point of comparison across expert offloading techniques. ***AllExp*** executes all experts on PIM [21, 43]. Although Stratum [43] targets monolithic 3D-Stackable DRAM rather than HBM-PIM, we reproduce its scheduling policy, which performs all expert computations in the decode phase using near-memory processing. PAPI [21] similarly explains that PIM with a larger number of processing units can efficiently handle expert computations. ***PIMoE*** assumes that all experts are assigned to PIM first and moves the most popular

expert from the busiest PIM channel to the GPU until the GPU execution time becomes larger than the PIM execution time [51].

Since a request yields one prefill phase followed by multiple decode phases, approximately 90% of stages in continuous batching correspond to decoding-only phases [27, 53]. Moreover, in modern inference systems, the prefill and decode phases are often executed on separate resources, a practice known as prefill-decode disaggregation [45, 56]. As a result, efficiently processing batches composed solely of decode-phase requests has become critical for practical deployments. Therefore, we mainly evaluate Sieve in the case where all requests in a batch are in the decode phase. Scenarios that mix prefill and decode phases are evaluated in Section 7.3.

7.2 Results

Evaluation Metrics for the Pareto Curve: Various parallelization strategies, including data parallelism, tensor parallelism, context parallelism, and expert parallelism, can be combined depending on whether a system targets high throughput or low latency. Therefore, rather than evaluating Sieve using either throughput or latency alone, we use a Pareto curve defined by interactivity (generated tokens per second per user) and throughput per GPU (generated tokens per second per GPU). Higher interactivity corresponds to lower latency, and points in the upper-right region represent efficient system performance.

Throughput and Interactivity Improvements by Sieve: Figure 9 illustrates the throughput (tokens per second) and interactivity of Sieve compared to prior methods across various batch sizes on *GPT-OSS* and *Qwen3.5*. Sieve consistently outperforms all baselines, delivering substantial gains in both total system throughput and per-user interactivity. Crucially, Sieve demonstrates superior system performance and scalability compared to the optimized *PIMoE* baseline. For small batch sizes, *PIMoE* and *AllExp* achieve performance comparable to Sieve, since most experts are memory-bound and assigning all experts to PIM is effective in this setting.

However, their performance starts to degrade with larger batch sizes, as the proportion of compute-bound experts increases. Sieve adaptively assigns such compute-bound experts to GPUs, achieving higher speedups than *PIMoE* and *AllExp* at larger batch sizes. On *Qwen3.5*, Sieve delivers up to a 26% (1.26×) improvement over *PIMoE* in both throughput and interactivity at $B = 256$. Similarly, on *GPT-OSS*, it yields a steady 11%-17% (1.11×-1.17×) throughput gain across moderate to large batch sizes ($B \geq 32$), with interactivity improvements reaching up to 1.25×. Ultimately, Sieve is the only approach that smoothly scales peak throughput at high loads while strictly satisfying interactivity service-level agreements (SLAs).

Analysis of Pareto Curve: Figure 9 reveals distinct performance trajectories for the baselines and Sieve. *NoExp*, which relies heavily on GPUs, exhibits an L-shaped curve, indicating suboptimal scaling with early throughput saturation. Conversely, executing all experts on the PIM (*AllExp*) yields an almost flat horizontal trajectory; it maintains interactivity, but throughput fails to scale beyond $B = 32$. Sieve and *PIMoE* achieve a curve closer to the ideal inverted-L frontier.

This behavior is strongly correlated with how the expert distribution evolves as the batch size increases. At highly constrained batch sizes ($B \leq 16$), the extreme sparsity of models like *Qwen3.5* reduces most expert computations to memory-bound operations. In this regime, confining execution to the PIM is strictly optimal, allowing Sieve and *PIMoE* to match *AllExp*'s high performance. However, as the batch size grows past 16 and approaches 32, the workload generates popular experts with higher arithmetic intensity. The key to achieving a curve closer to the ideal inverted-L trajectory is recognizing this transition point. By selectively offloading these hot experts to the GPU at $B \geq 32$, Sieve surpasses the throughput ceiling at which *AllExp* saturates.

Architectural Advantages of Sieve: While both Sieve and *PIMoE* achieve curves closer to the ideal inverted-L trajectory compared to other baselines, Sieve achieves significantly higher peak throughput (a $\sim$ 1.26× improvement for *Qwen3.5*) by addressing two critical architectural blind spots. First, *PIMoE* balances expert placement by evaluating isolated expert execution times but ignores attention computation on the PIM, whose cost grows rapidly at larger batch sizes. Second, in real-world MoE inference, an optimal schedule can only be achieved when inter-GPU communication, compute, and memory demands are considered together. *PIMoE* overlooks the overhead of communication across GPUs and overloads the PIM, extending PIM execution beyond the latency of network transfers and thereby limiting overall throughput.

In contrast, Sieve explicitly incorporates both network communication costs and attention overheads into its scheduling policy. As a result, Sieve achieves higher throughput and interactivity than prior work in both multi-GPU (`Qwen3.5` and *GPT-OSS* in Figure 9) and single-GPU settings (*Qwen3* in Figure 9). In single-GPU settings, Sieve achieves up to a 1.6× improvement in throughput and interactivity over *PIMoE* by accounting for the attention overheads in the scheduling policy. In multi-GPU settings, Sieve reduces PIM latency below the communication threshold by accounting for network communication costs and attention overheads, unlocking an additional 1.3× improvement in throughput and interactivity over *PIMoE* on *Qwen3.5* and *GPT-OSS*. To conclude, Sieve effectively shifts the Pareto frontier in both single- and multi-GPU setups, guaranteeing that the system processes exponentially more tokens globally without penalizing the responsiveness of individual user requests.

Parallelism Across PIM Channels: As discussed in Section 6.2, Sieve employs tensor parallelism across PIM channels within a GPU, while *PIMoE* uses expert parallelism. The arithmetic intensity disparity across experts leads to significant fluctuations in PIM channel utilization in *PIMoE*, as illustrated in Figure 10; certain channels experience heavy workloads while others remain underutilized. Furthermore, since commercial PIM architectures lack direct data transfer between PIM channels [4], the applicability of the Expert Parallelism Load Balancer across PIM channels in *PIMoE* is limited, making it difficult to mitigate this hardware-level inefficiency. However, since Sieve utilizes tensor parallelism across PIM channels, every channel is uniformly utilized as long as memory-bound experts are executed on PIM. This approach effectively decouples PIM channel utilization from the expert distribution.

7.3 Colocated Prefill-Decode Requests

Previous results demonstrate that Sieve is a promising solution for efficient MoE serving within Prefill-Decode (PD) disaggregation,

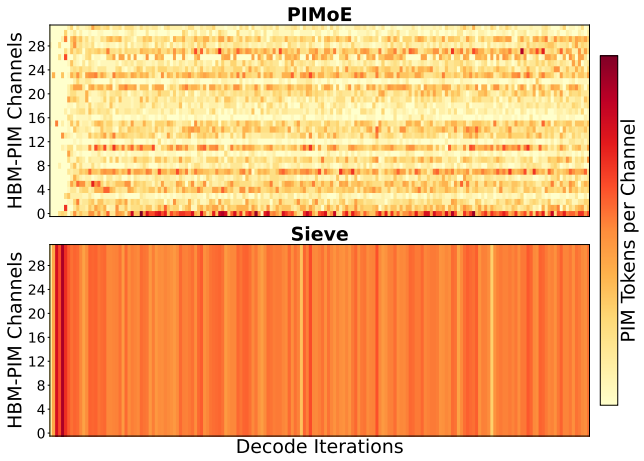

Figure 10: Utilization of PIM channels when running *GPT-OSS* with 4 B200 GPUs equipped with HBM-PIM stacks. A darker color means a PIM channel has higher utilization.

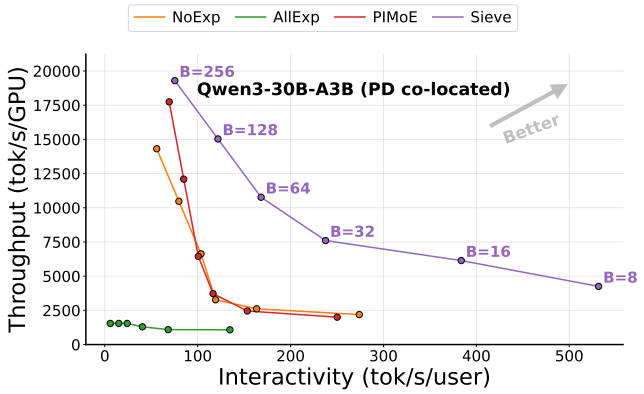

Figure 11: Evaluation of throughput and interactivity achieved by Sieve, *NoExp*, *AllExp*, and *PIMoE* for *Qwen-3* under colocated prefill-decode requests.

although PD disaggregation may not always represent practical deployments. To address this, we evaluate Sieve under colocated PD in *Qwen3*, where each batch contains both prefill-phase and decode-phase requests. State-of-the-art LLM frameworks such as vLLM [30] limit batches to at most two prefill-phase requests because queries often exceed 1024 tokens. To stress-test Sieve, however, we construct extreme cases with up to eight prefill-phase requests per batch for $B \geq 64$, and up to two prefill-phase requests per batch for $B \leq 32$.

As shown in Figure 11, Sieve consistently outperforms all prior methods under colocated prefill-decode requests. Even in extreme cases where eight prefill-phase requests coexist in a batch, Sieve achieves 2.4× ($B = 16$) and 2.3× ($B = 32$) speedups compared to *NoExp*, respectively. In contrast, both *AllExp* and *PIMoE* exhibit noticeable slowdowns as the number of prefill-phase requests increases, often performing worse than *NoExp*. This degradation occurs because a higher fraction of prefill requests increases the probability of executing compute-bound GEMM experts on PIM, a situation that Sieve effectively avoids. As there are eight prefill requests at $B \geq 64$, the number of compute-bound experts becomes larger. Moreover, Sieve achieves greater throughput gains at $B \geq 64$ compared to $B \leq 32$ as shown in Figure 11. Consequently, Sieve can efficiently support MoE serving in both colocated prefill-decode and disaggregated prefill-decode scenarios by maintaining balanced utilization between PIM and GPU.

8 Related Work

Recent research has extensively explored PIM to mitigate the memory wall in LLM inference [21, 22, 32, 44]. A dominant direction is to offload memory-bound operators, such as attention computation, to PIM while keeping compute-bound operators such as dense FFNs on the xPU [21, 22, 32, 44]. These studies show the effectiveness of heterogeneous xPU–PIM execution for dense transformers, but largely assume that the offloaded portion is known a priori from the model structure or memory layout.

A second line of work considers MoE-aware heterogeneous execution. Prior studies observe that MoE layers exhibit varying arithmetic intensity and advocate heterogeneous execution across xPU and PIM [51, 53]. However, prior approaches rely on static or coarse-grained placement policies that do not fully capture the highly imbalanced expert distribution observed in recent sparse MoE models [51, 53].

Sieve is complementary to orthogonal efforts that improve the PIM substrate or target bottlenecks, such as long-context attention or KV-cache management [23, 29, 37, 46]. These approaches can be combined with Sieve because they do not address the same problem of dynamically partitioning MoE expert computations between GPU and PIM based on the runtime expert distribution. In contrast, Sieve targets modern and future MoE serving scenarios and provides a lightweight runtime scheduler that maps unpopular, memory-bound experts to PIM while keeping popular experts on the xPU.

9 Conclusion

This paper presents a scheduler for MoE models running on multi-GPU systems with PIM, accounting for both the arithmetic intensity disparity and inter-GPU communication incurred by expert parallelism. Furthermore, we propose Sieve, a framework for the proposed scheduler, which accelerates modern MoE models on PIM-enabled systems by efficiently coordinating operations on GPUs and PIM. Our evaluations on state-of-the-art MoE models demonstrate throughput and interactivity gains over prior PIM systems for MoE.

References

[1] [n. d.]. GLM-4.5-FP8 Model Card. https://huggingface.co/zai-org/GLM-4.5-FP8. Accessed: 2025-11-15.
[2] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).
[3] Artificial Analysis. 2025. Artificial Analysis. https://artificialanalysis.ai. Accessed: 2025-11-03.
[4] Daehyeon Baek, Soojin Hwang, and Jaehyuk Huh. 2024. pSyncPIM: Partially Synchronous Execution of Sparse Matrix Operations for All-Bank PIM Architectures. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 354–367.

[5] Baidu-ERNIE-Team. 2025. ERNIE 4.5 Technical Report. https://yiyan.baidu.com/blog/publication/ERNIE_Technical_Report.pdf

[6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[7] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*.

[8] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, et al. 2014. Dadiannao: A machine-learning supercomputer. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 609–622.

[9] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. *ACM SIGARCH computer architecture news* 44, 3 (2016), 367–379.

[10] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).

[11] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv:2401.06066 [cs.CL] https://arxiv.org/abs/2401.06066

[12] Bill Dally. 2023. Hardware for deep learning. In *2023 IEEE Hot Chips 35 Symposium (HCS)*. IEEE Computer Society, 1–58.

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[14] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning*. PMLR, 5547–5569.

[15] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[16] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, Andy Jones, Sam Bowman, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Nelson Elhage, Sheer El-Showk, Stanislav Fort, Zac Hatfield-Dodds, Tom Henighan, Danny Hernandez, Tristan Hume, Josh Jacobson, Scott Johnston, Shauna Kravec, Catherine Olsson, Sam Ringer, Eli Tran-Johnson, Dario Amodei, Tom Brown, Nicholas Joseph, Sam McCandlish, Chris Olah, Jared Kaplan, and Jack Clark. 2022. Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned. arXiv:2209.07858 [cs.CL] https://arxiv.org/abs/2209.07858

[17] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. 2024. Ai and memory wall. *IEEE Micro* 44, 3 (2024), 33–39.

[18] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi Iyer, and Reetuparna Das. 2025. PIM is all you need: A CXL-enabled GPU-free system for large language model inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 862–881.

[19] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[20] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and TN Vijaykumar. 2020. Newton: A DRAM-maker's accelerator-in-memory (AiM) architecture for machine learning. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 372–385.

[21] Yintao He, Haiyu Mao, Christina Giannoula, Mohammad Sadrosadati, Juan Gómez-Luna, Huawei Li, Xiaowei Li, Ying Wang, and Onur Mutlu. 2025. Papi: Exploiting dynamic parallelism in large language model decoding with a processing-in-memory-enabled computing system. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 766–782.

[22] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 722–737.

[23] Yang Hong, Junlong Yang, Bo Peng, and Jianguo Yao. 2026. REPA: Re configurable P IM for the Joint A cceleration of KV Cache Offloading and Processing. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1622–1639.

[24] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG] https://arxiv.org/abs/2401.04088

[25] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.

[26] Jin Hyun Kim, Shin-Haeng Kang, Sukhan Lee, Hyeonsu Kim, Yuhwan Ro, Seungwon Lee, David Wang, Jihyun Choi, Jinin So, YeonGon Cho, et al. 2022. Aquabolt-XL HBM2-PIM, LPDDR5-PIM with in-memory processing, and AXDIMM with acceleration buffer. *IEEE Micro* 42, 3 (2022), 20–30.

[27] Jin Hyun Kim, Yuhwan Ro, Jinin So, Sukhan Lee, Shin-haeng Kang, YeonGon Cho, Hyeonsu Kim, Byeongho Kim, Kyungsoo Kim, Sangsoo Park, et al. 2023. Samsung pim/pnm for transfmer based ai: Energy efficiency on pim/pnm cluster. In *2023 IEEE Hot Chips 35 Symposium (HCS)*. IEEE Computer Society, 1–31.

[28] Wonung Kim, Yubin Lee, Yoonsung Kim, Jinwoo Hwang, Seongryong Oh, Jiyong Jung, Aziz Huseynov, Woong Gyu Park, Chang Hyun Park, Divya Mahajan, et al. 2025. Pimba: A Processing-in-Memory Acceleration for Post-Transformer Large Language Model Serving. *arXiv preprint arXiv:2507.10178* (2025).

[29] Hyucksung Kwon, Kyungmo Koo, Janghyeon Kim, Woongkyu Lee, Minjae Lee, Gyeonggeun Jung, Hyungdeok Lee, Yousub Jung, Jaehan Park, Yosub Song, et al. 2026. PIMphony: Overcoming Bandwidth and Capacity Inefficiency in PIM-Based Long-Context LLM Inference System. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–21.

[30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

[31] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, et al. 2022. System architecture and software stack for GDDR6-AiM. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE, 1–25.

[32] Hyojung Lee, Daehyeon Baek, Jimyoung Son, Jieun Choi, Kihyo Moon, and Minsung Jang. 2025. PAISE: PIM-Accelerated Inference Scheduling Engine for Transformer-based LLM. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1707–1719.

[33] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, et al. 2021. Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 43–56.

[34] Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, et al. 2022. A 1ynm 1.25 V 8Gb, 16Gb/s/pin GDDR6-based accelerator-in-memory supporting 1TFLOPS MAC operation and various activation functions for deep-learning applications. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. IEEE, 1–3.

[35] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).

[36] Cong Li, Zhe Zhou, Size Zheng, Jiaxi Zhang, Yun Liang, and Guangyu Sun. 2024. Specpim: Accelerating speculative inference on pim-enabled system via architecture-dataflow co-exploration. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 950–965.

[37] Sixu Li, Yuzhou Chen, Chaojian Li, Yonggan Fu, Zheng Wang, Zhongzhi Yu, Haoran You, Zhifan Ye, Wei Zhou, Yongan Zhang, et al. 2025. ORCHES: Orchestrated Test-Time-Compute-based LLM Reasoning on Collaborative GPU-PIM HEterogeneous System. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 476–489.

[38] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*.

[39] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[40] Haocong Luo, Yahya Can Tuğrul, F. Nisa Bostancı, Ataberk Olgun, A. Giray Yağlıkçı, , and Onur Mutlu. 2023. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator.

[41] Meta AI. 2025. The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal Models. https://ai.meta.com/blog/llama-4-multimodal-intelligence/

[42] NVIDIA. 2025. NVIDIA DGX B200 Datasheet. https://resources.nvidia.com/en-us-dgx-systems/dgx-b200-datasheet.

[43] Yue Pan, Zihan Xia, Po-Kai Hsu, Lanxiang Hu, Hyungyo Kim, Janak Sharda, Minxuan Zhou, Nam Sung Kim, Shimeng Yu, Tajana Rosing, et al. 2025. Stratum: System-Hardware Co-Design with Tiered Monolithic 3D-Stackable DRAM for Efficient MoE Serving. *arXiv preprint arXiv:2510.05245* (2025).

[44] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. Attacc! unleashing the power of pim for batched transformer-based generative model inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 103–119.

[45] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.

[46] Derrick Quinn, E Ezgi Yücel, Jinkwon Kim, José F Martínez, and Mohammad Alian. 2025. LongSight: Compute-Enabled Memory to Accelerate Large-Context LLMs via Sparse Attention. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 34–48.

[47] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).

[48] GLM-4.5 Team. 2025. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. https://arxiv.org/abs/2508.06471

[49] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. 2025. Kimi K2: Open Agentic Intelligence. *arXiv preprint arXiv:2507.20534* (2025).

[50] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2019. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).

[51] Lizhou Wu, Haozhe Zhu, Siqi He, Xuanda Lin, Xiaoyang Zeng, and Chixiao Chen. 2025. PIMoE: Towards efficient MoE transformer deployment on NPU-PIM system through throttle-aware task offloading. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.

[52] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[53] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1429–1443.

[54] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, et al. 2025. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. *Proceedings of Machine Learning and Systems* 7 (2025).

[55] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.

[56] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 11, 18 pages.

[57] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, et al. 2025. Megascale-infer: Serving mixture-of-experts at scale with disaggregated expert parallelism. *arXiv preprint arXiv:2504.02263* (2025).